

Relatório da 1ª Entrega do Projeto

Tema: Gestão de Máquinas de Vending

1. Descrição Breve do Cenário a ser estudado no Projeto

O projeto explora o cenário de gestão de máquinas de vending, abrangendo desde o armazenamento e reposição de produtos até à gestão de transações e inventário em tempo real. Este cenário é frequentemente encontrado em empresas que operam redes de máquinas de venda automática, oferecendo produtos como bebidas, snacks e outros itens de conveniência em locais estratégicos como escritórios, escolas, hospitais e transportes públicos. O objetivo é desenvolver uma solução eficiente e escalável que permita:

1. Gestão Centralizada:

- Controlar remotamente o inventário e as vendas de máquinas espalhadas por diferentes regiões.

2. Otimização de Recursos:

- Planejar rotas de reabastecimento com base em dados atualizados de inventário, reduzindo custos operacionais e aumentando a eficiência.

3. Análise de Desempenho:

- Monitorar o desempenho de vendas em diferentes localizações para apoiar a tomada de decisões baseadas em dados.

4. Integração com Sistemas de Pagamento:

- Registrar transações automáticas, incluindo métodos de pagamento modernos como carteiras digitais e outros sistemas cashless.

5. Melhoria na Experiência do Cliente:

- Garantir que os produtos estejam sempre disponíveis e manter as máquinas em bom estado de funcionamento, melhorando a satisfação dos clientes.

Este projeto também aborda desafios como a segurança dos dados de transações, a manutenção preventiva das máquinas e a integração com sistemas externos de relatórios ou serviços logísticos. Além disso, busca-se criar uma base sólida para expansão futura, com potencial de incluir funcionalidades como monitorização em tempo real e recomendações de inventário baseadas em inteligência artificial.

2. Modelo de Dados

O modelo de dados é uma parte essencial de qualquer sistema de informação, pois define como as informações são organizadas, armazenadas e acessadas. No contexto do projeto de gestão de máquinas de vending, o modelo de dados foi projetado para refletir a complexidade e a relação entre diferentes entidades, garantindo escalabilidade, integridade e facilidade de uso.

Estrutura Geral do Modelo

O modelo de dados deste sistema foi desenvolvido utilizando uma abordagem relacional, implementada no banco de dados PostgreSQL. Ele inclui as seguintes tabelas principais:

1. Categoria (Category)

- Representa as categorias dos produtos vendidos, como "Bebidas", "Snacks" e "Doces".
- Campos principais:
 - `id` (chave primária): Identificador único da categoria.
 - `name`: Nome da categoria.
 - `description`: Descrição detalhada da categoria.

2. Produto (Product)

- Representa os produtos vendidos pelas máquinas.
- Relaciona-se com a tabela **Categoria** através de uma chave estrangeira (`category_id`).
- Campos principais:
 - `id` (chave primária): Identificador único do produto.
 - `name`: Nome do produto.
 - `description`: Descrição detalhada do produto.
 - `price`: Preço do produto.
 - `category_id`: Referência à categoria do produto.
 - `is_active`: Indica se o produto está ativo ou não.

3. Máquina de Vending (Vending Machine)

- Representa as máquinas de vending que armazenam os produtos.
- Campos principais:
 - `id` (chave primária): Identificador único da máquina.
 - `location`: Localização física da máquina.
 - `description`: Descrição detalhada da máquina.

4. Inventário (Inventory)

- Representa os produtos armazenados em cada máquina de vending e suas respectivas quantidades.

- Relaciona-se com as tabelas **Máquina de Vending** e **Produto**.
- Campos principais:
 - `id` (chave primária): Identificador único do registro de inventário.
 - `machine_id`: Referência à máquina onde o produto está armazenado.
 - `product_id`: Referência ao produto armazenado.
 - `quantity`: Quantidade do produto disponível.

5. Transações (Transactions)

- Regista as vendas realizadas pelas máquinas de vending.
- Relaciona-se com as tabelas **Máquina de Vending** e **Produto**.
- Campos principais:
 - `id` (chave primária): Identificador único da transação.
 - `machine_id`: Referência à máquina onde a transação ocorreu.
 - `product_id`: Referência ao produto vendido.
 - `quantity`: Quantidade do produto vendido.
 - `total_price`: Valor total da transação.
 - `transaction_date`: Data e horário da transação.

6. Usuários (Users)

- Representa os usuários do sistema, incluindo administradores e operadores.
- Campos principais:
 - `id` (chave primária): Identificador único do usuário.
 - `username`: Nome de usuário único.
 - `password`: Senha criptografada do usuário.
 - `role`: Função do usuário (e.g., "admin" ou "operator").

Relacionamentos entre Tabelas

O modelo de dados estabelece os seguintes relacionamentos:

- **Categoria e Produto:** Uma categoria pode ter vários produtos (relacionamento 1:N).
- **Produto e Inventário:** Um produto pode estar em várias máquinas, mas cada registo de inventário é único para uma combinação de produto e máquina (relacionamento N:M implementado através da tabela `Inventory`).
- **Máquina de Vending e Inventário:** Cada máquina pode ter vários produtos em seu inventário (relacionamento 1:N).
- **Transações e Produto/Máquina de Vending:** Cada transação está associada a uma máquina e a um produto específico (relacionamento N:M).

3. Descrição das Principais Operações a Implementar (Endpoints)

Abaixo estão descritos os endpoints principais que deverão ser implementados no sistema, agrupados pelas funcionalidades relacionadas:

1. Categoria (Category):

- GET /api/category: Retorna a lista de categorias.
- GET /api/category/{id}: Retorna uma categoria pelo seu ID.
- POST /api/category: Cria uma nova categoria.
- PUT /api/category/{id}: Atualiza os dados de uma categoria existente.
- DELETE /api/category/{id}: Remove uma categoria.

2. Produto (Product):

- GET /api/product: Retorna a lista de produtos.
- GET /api/product/{id}: Retorna um produto pelo seu ID.
- POST /api/product: Cria um novo produto associado a uma categoria.
- PUT /api/product/{id}: Atualiza os dados de um produto existente.
- DELETE /api/product/{id}: Remove um produto.

3. Máquina de Vending (Vending Machine):

- GET /api/vending-machine: Retorna a lista de máquinas de vending.
- GET /api/vending-machine/{id}: Retorna uma máquina de vending pelo seu ID.
- POST /api/vending-machine: Cria uma nova máquina de vending.
- PUT /api/vending-machine/{id}: Atualiza os dados de uma máquina existente.
- DELETE /api/vending-machine/{id}: Remove uma máquina de vending.

4. Inventário (Inventory):

- GET /api/inventory: Retorna o inventário completo.
- GET /api/inventory/{machineId}: Retorna o inventário de uma máquina específica.
- POST /api/inventory: Adiciona ou atualiza produtos no inventário de uma máquina.

5. Transações (Transactions):

- GET /api/transactions: Retorna o histórico de transações.
- GET /api/transactions/{id}: Retorna os detalhes de uma transação específica.
- POST /api/transactions: Regista uma nova transação.

6. Utilizadores (Users):

- GET /api/users: Retorna a lista de utilizadores.
- GET /api/users/{id}: Retorna os detalhes de um utilizador.
- POST /api/users: Cria um novo utilizador.
- PUT /api/users/{id}: Atualiza os dados de um utilizador.
- DELETE /api/users/{id}: Remove um utilizador.

4. Arquitetura

Operações a Implementar em Web Services SOAP (XML)

As operações SOAP são orientadas a transações e serão utilizadas para integrações com sistemas externos e operações mais sensíveis. Os principais endpoints incluem:

1. Registo de Transações:

- Operações para registar e consultar transações de vendas realizadas nas máquinas.
- Suporte a integrações com sistemas de auditoria ou faturamento externo.

2. Consulta de Inventário:

- Disponibiliza informações detalhadas sobre o inventário de cada máquina.

Operações a Implementar em Web API RESTful

As operações RESTful fornecem uma interface flexível para o consumo pelos clientes front-end e por outros serviços. Alguns exemplos de endpoints principais são:

1. Gestão de Produtos e Categorias:

- Manipulação de produtos, categorias e inventário.

2. Consulta de Vendas e Relatórios:

- Oferece histórico de vendas e relatórios filtrados por data, local ou produto.

Implementação e Acesso ao Banco de Dados

- A implementação utilizará comandos SQL diretamente, sem ORM, garantindo maior controle e eficiência.
- As queries serão executadas utilizando bibliotecas como `Npgsql` no C# para conectar ao PostgreSQL.
- A validação e formatação dos dados serão realizadas no código antes de enviar as queries.

Serviços Externos Consumidos

1. Autenticação e Autorização:

- Serviço externo para autenticação segura (e.g., OAuth ou JWT).

2. Pagamento Online:

- Integração com plataformas de pagamento para futuras transações realizadas em aplicações móveis ou web.

3. Notificações:

- Integração com APIs de envio de e-mails ou SMS para alertas sobre inventário ou falhas.

Conclusão

Nesta 1ª entrega, foi definido o escopo do projeto, o modelo de dados, e as principais operações RESTful e SOAP. A arquitetura proposta assegura flexibilidade, eficiência e integração com serviços externos, utilizando uma infraestrutura moderna e escalável com CockroachDB e PostgreSQL. O sistema está pronto para evoluir nas próximas etapas com foco na implementação dos endpoints e integração de serviços externos.